

Урок 1. Корутины. Введение

25 ноября 2019

В этом уроке начнем разбираться, что такое корутина и suspend функция.

Сложная тема

Не раз я слышал мнение, что официальная документация по корутинам сложна и представляет собой примерно такое:



Я, пожалуй, соглашусь с этим мнением. Нас сразу грузят билдерами, скоупами и suspend функциями. Говорят, что их надо использовать так-то и так-то и будет нам счастье. И вроде даже объясняют, что это такое, но особо понятнее не становится. Но даже несмотря на это я рекомендую вам посмотреть эту документацию, чтобы получить хотя бы примерное представление, что такое корутины и зачем они нужны.

В защиту авторов документации я должен сказать, что тема действительно очень сложна для объяснения. Для меня корутины по сложности легко обошли такие непростые темы, как Dagger или Backpressure в RxJava. Я потратил кучу времени, чтобы разобраться, что же такое корутины и как они работают. Читал официальные доки, делал примеры, читал статьи, смотрел видео. И все равно у меня оставалось ощущение, что я не понимаю их до конца сам, а значит не могу объяснить другим. Пришлось прибегнуть к последнему, самому надежному и самому сложному средству - лезть в исходники. Врагу не пожелаю туда соваться, но в итоге я таки пробился через эти дебри и постепенно пазл собрался.

В этом курсе я собираюсь достаточно подробно осветить тему корутин, используя для этого все то, что мне удалось раскопать. Я буду шаг за шагом расписывать отдельные кусочки пазла и периодически собирать эти кусочки в большие куски, объясняя очередную тему. В итоге у вас должна сложиться общая картина. Иногда может показаться, что я слишком ухожу в дебри объяснений, и эта информация не нужна вовсе, и давайте лучше сразу с боевых примеров начнем! Но тут я исхожу из своего опыта. Мне удалось полностью понять примеры только после того, как я раскопал внутренности. И сейчас моя цель - сделать так, чтобы вы, посмотрев на код с корутинами, могли точно сказать, как он себя поведет. А для этого нужно понять корутины изнутри.

Поэтому первые несколько уроков будут состоять только из объяснений. Я подробно расскажу о том, во что превращается корутина при преобразовании Kotlin кода в Java. А также о том, почему suspend функция не блокирует поток. Об этом будут первые 5 уроков. А уже после этого пойдут более интересные и приближенные к практике темы: Scope, Context, Job и т.п.

Рекомендую не пропускать уроки и идти по ним последовательно, чтобы в последующих уроках все было понятно. А если какой-то урок или раздел можно будет пропустить, я явно напишу об этом.

Краткий вариант уроков

В этом курсе я буду использовать новую технику подачи материала. Если урок содержит много теории и объяснений, то вы сможете прочитать его в одном из двух вариантов: подробном или кратком.

Подробный вариант подходит для тех, кто в теме новичок. В этом случае требуется максимально подробное объяснение темы. Одна и та же мысль может быть рассмотрена несколько раз с разных сторон для лучшего понимания. Читать надо долго и вдумчиво. В этом стиле написано большинство моих уроков на сложные темы.

Краткий вариант подходит для тех, кто уже хоть немного, но знаком с темой. А также тех, кто ранее уже прочел подробный вариант и зашел просто освежить знания или подсмотреть какой-то момент. Тут не будет подробных объяснений и долгого хождения вокруг темы. Все лаконично и тезисно.

Выбирайте вариант, который подходит вам. Можете сразу попробовать краткий вариант для экономии времени. Если видите, что информации недостаточно, и нужно более вдумчивое объяснение, то заваривайте кружку чая, берите плед и открывайте подробный вариант)

Эта опция может некорректно работать на мобильной версии сайта. Я уже написал в техподдержку создателям, жду ответа. В случае возникновения проблем используйте десктопную версию.

Корутина

Непросто подобрать описание, что такое корутина. В самом начале обычно рассматривается такой простой пример:

```
1 launch {
2     delay(1000L)
3     println("World!")
4 }
```

Этот и последующие несколько примеров выдернуты из контекста и не будут работать без дополнительного кода. Но в целях изучения теории они нам подходят. Практика будет позже.

Здесь launch - это билдер корутины, которому передается небольшой блок кода:

```
1 delay(1000L)
2 println("World!")
```

Мы еще рассмотрим очень подробно и по шагам, что происходит под капотом билдера. А пока что нам надо знать только то, что билдер упакует переданный ему блок кода в корутину и запустит ее. Если необходимо, то с помощью входных параметров мы можем указать билдеру, в каком потоке необходимо выполнить этот код.

Если очень упрощать, то можно сказать, что билдер - это экзекьютор, который оборачивает блок кода в Runnable и запускает этот Runnable на выполнение. Т.е. корутину можно представить так:

```
1 executor.submit {
2     delay(1000L)
3     println("World!")
4 }
```

Казалось бы, все, тему можно закрывать. Корутина - это просто очередной механизм для выполнения асинхронных операций. Но не все так просто.

Корутина имеет ряд особенностей. Например - Job. Когда мы запускаем корутину, мы можем получить Job, как результат запуска билдера:

```
1 val job = launch {
2     delay(1000L)
3     println("World!")
4 }
```

Это дает нам некоторые возможности по управлению корутиной. Мы можем сделать выполнение кода отложенным (LAZY) и стартовать его позже, когда понадобится. Или в любой момент времени можно будет отменить выполнение. Также можно запускать корутину внутри корутины. Их джобы будут связаны между собой отношениями Parent-Child, что является отдельным механизмом, который влияет на обработку ошибок и отмену корутин. Все это мы еще подробно рассмотрим в дальнейших уроках.

А пока что нас интересует одна из самых важных особенностей корутин - suspend функции.

Suspend функция

Чтобы лучше понять, что это такое, поговорим про потоки. Поток выполнения корутины - это вовсе не обязательно фоновый поток. Им вполне может быть и main поток. В Android мы часто будем запускать корутины в main потоке. И, как мы все знаем, этот поток нельзя блокировать.

В примере выше мы в корутине используем suspend функцию delay. Эта функция приостановит выполнение кода на 1000 мсек, затем выполнение кода продолжится и напишет в лог слово World. Вроде ничего не обычного. Но обратите внимание, я написал "приостановит выполнение кода", а не "заблокирует поток".

В этом и есть ключевая особенность suspend функции. Она не блочит поток. А значит мы можем эту корутину запускать в main потоке. Функция delay приостановит выполнение кода на 1 секунду, но не заблокирует main поток.

Давайте рассмотрим пример, более приближенный к реальности. Например, загрузка файла.

Код без всяких корутин и suspend функций будет таким:

```
1 val url = buildUrl()
2
3 // long synchronous function
4 download(url)
5
6 toast("File is downloaded")
```

Мы генерируем URL файла и передаем его в функцию download, которая выполнит загрузку. После этого мы выводим Toast-сообщение о том, что файл загружен.

Функция download - синхронная, а значит заблокирует поток, в котором будет выполнена. Но нам хотелось бы выполнить этот код в UI потоке, потому что функция toast может быть выполнена только в нем. Чтобы разрешить это противоречие, нам придется сделать функцию download асинхронной и toast поместить в колбэк, который будет выполнен по завершении загрузки.

```
1 val url = buildUrl()
2
3 // long asynchronous function
4 download(url) {
5     toast("File is downloaded")
6 }
```

Т.е. при запуске тяжелого кода в main потоке нам приходится выбирать из двух зол: блокировка потока или колбэк. Если функция download синхронна - то поток будет заблокирован. А если она асинхронна, то нам придется добавлять колбэк, чтобы выполнить последующий код.

А хорошо было бы сделать так, чтобы долго работающая функция не блокировала поток, но при этом код, который находится после функции, был выполнен по завершении загрузки без всяких колбэков. Использование suspend функций в корутинах позволяют нам сделать это.

Перепишем код с использованием корутины и suspend функции:

```
1 launch {
2     val url = buildUrl()
3
4     download(url) //suspend function
5
6     toast("File is downloaded")
7 }
```

Эта корутина не заблокирует поток, в котором будет запущен ее код. Т.е. его можно запустить даже в main потоке. Функция download загрузит файл в отдельном потоке, а toast будет выполнен только после того, как download отработает.

Это кажется магией, но не забывайте, что Kotlin код будет преобразован в Java классы. И во время этих преобразований будет использован механизм Continuation. Он в сочетании с suspend функциями реализует колбэк, который обеспечит выполнение toast только после того, как загрузка файла будет завершена.

О том, как именно это происходит, подробно поговорим в следующем уроке.

Кратко

Корутина

В самом начале документации и статей обычно рассматривается такой простой пример:

```
1 launch {
2     delay(1000L)
3     println("World!")
4 }
```

Здесь launch - это билдер корутины, которому передается блок кода. Билдер упакует переданный ему блок кода в корутину и запустит ее.

Когда мы запускаем корутину, мы можем получить Job, как результат запуска билдера:

```
1 val job = launch {
2     delay(1000L)
3     println("World!")
4 }
```

Это дает нам некоторые возможности по управлению корутиной. Например, мы можем сделать выполнение кода отложенным (lazy) и стартовать его позже, когда понадобится. Или в любой момент времени можно будет отменить выполнение. Также можно запускать корутину внутри корутины. Их джобы будут связаны между собой отношениями Parent-Child, что является отдельным механизмом, который влияет на обработку ошибок и отмену корутин.

Suspend функция

В примере выше мы в корутине используем suspend функцию delay. Эта функция приостановит выполнение кода на 1000 мсек, затем выполнение кода продолжится и напишет в лог слово World. При этом delay не заблокирует поток. В этом и есть ключевая особенность suspend функций. А значит мы можем эту корутину запускать в main потоке.

Давайте рассмотрим пример, более приближенный к реальности. Например, загрузка файла.

Код без всяких корутин и suspend функций будет таким:

```
1 val url = buildUrl()
2
3 // long synchronous function
4 download(url)
5
6 toast("Url is downloaded")
```

Мы генерируем URL файла и передаем его в функцию download, которая выполнит загрузку. После этого мы выводим Toast-сообщение о том, что файл загружен.

Функция download - синхронная, а значит заблокирует поток, в котором будет выполнена. Но нам хотелось бы выполнить этот код в UI потоке, потому что функция toast может быть выполнена только в нем. Чтобы разрешить это противоречие, нам придется сделать функцию download асинхронной и toast поместить в колбэк, который будет выполнен по завершении загрузки.

```
1 val url = buildUrl()
2
3 // long asynchronous function
4 download(url) {
5     toast("Url is downloaded")
6 }
```

Использование suspend функций в корутинах позволяют нам выполнять долгие асинхронные функции без колбэков:

```
1 launch {
2     val url = buildUrl()
3
4     download(url) //suspend function
5
6     toast("Url is downloaded")
7 }
```

Эта корутина не заблокирует поток, в котором она будет запущена. А toast будет выполнен только после того, как download отработает. Это достигается с помощью механизма Continuation, который используется при преобразовании Kotlin кода в Java классы.

О том, как именно это происходит, подробно поговорим в следующем уроке.